SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Operating Systems
COURSE CODE: CSA-0409

COURSE OUTCOME COVERED

CO3: Evaluate memory management mechanisms such as linking, paging, and segmentation to determine their effectiveness for different system requirements

Assessment Tool Weightage: 30%

QUESTIONS: 10 **Total Marks: 50**

CASE STUDY:

11.

Explain the concept of **virtual memory** and its importance in modern operating systems. Describe how it allows execution of programs larger than the available physical memory.

12.

Describe the **page fault handling process** in detail. Explain each step involved when a page is not found in memory and how the operating system resolves the fault.

13.

Explain the need for **page replacement algorithms** in virtual memory systems. Describe the working of the **FIFO page replacement algorithm** with a suitable example.

14.

Discuss the concepts of **linking and loading** in detail. Explain the differences between static linking and dynamic linking, and describe how loading prepares a program for execution.

15.

Analyze how the combination of **paging, segmentation, TLB, and virtual memory** improves system performance and memory utilization in the given case study. Provide a detailed explanation.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided

Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

ANSWERS:

1. Explain the concept of **virtual memory** and its importance in modern operating systems. Describe how it allows execution of programs larger than the available physical memory.

CASE STUDY: Virtual Memory in a Student's Laptop

Situation:

Imagine a student has a laptop with 4 GB RAM.

The student opens these applications at the same time:

- Google Chrome
- Android Studio
- MS Word
- Music player

Together, these applications need around 6 GB of memory.

But the laptop has only 4 GB RAM.

Problem:

The student may think:

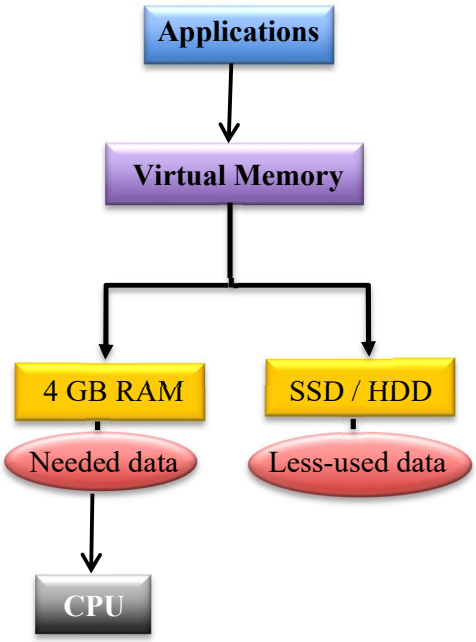
"If my laptop has only 4 GB RAM, how can it run programs that need 6 GB?"

This is where virtual memory is used.

How Virtual Memory Helps:

The operating system does not put the entire programs into RAM.

Instead, it keeps the currently needed parts in RAM and temporarily keeps the less-used parts on the SSD/HDD.



Simple Example:

- Suppose the student is currently using Android Studio.
- The OS keeps the important Android Studio pages in RAM.
- Some Chrome pages that are not currently being used can be moved to the SSD.
- Later, when the student switches back to Chrome, the OS brings the required pages back into RAM.
- This is called paging, and if the required page is not in RAM, it causes a page fault.

Why Is It Important?

Virtual memory helps the student:

- Run multiple applications.
- Run programs larger than physical RAM.
- Use RAM efficiently.
- Keep applications running without immediately running out of memory.

What If Virtual Memory Was Not Available?

Without virtual memory, the laptop would have difficulty running applications requiring more memory than the available RAM.

The operating system might have to close or stop some applications.

Conclusion:

Virtual memory acts like an extension of RAM. It allows a computer to run large programs and multiple applications even when physical RAM is limited. It does this by keeping currently needed data in RAM and storing less-used data on secondary storage.

---

2. Describe the **page fault handling process** in detail. Explain each step involved when a page is not found in memory and how the operating system resolves the fault.

CASE STUDY: Page Fault Handling

Situation:

Imagine a student is using Google Chrome. Chrome needs Page 5, but Page 5 is not currently available in RAM. It is stored on the SSD.

When the CPU tries to access Page 5, a page fault occurs.

Step-by-Step Process:

1. CPU Requests a Page:

The CPU generates a logical address containing:

- Page number
- Offset

For example:

Page Number = 5

Offset = 20

2. Page Table is Checked:

The operating system checks the page table to see whether Page 5 is in RAM.

3. Page Fault Occurs:

- Since the required page is not in RAM, the hardware generates a page fault trap.
- The current process is temporarily stopped, and control is given to the operating system.

4. OS Finds the Page:

The operating system locates Page 5 in secondary storage (SSD/HDD).

SSD

├── Page 1

├── Page 3

├── Page 5 ← Required page

└── Page 8

5. OS Finds a Free Frame:

- The OS checks RAM for an empty frame.
- If a free frame exists:
  The required page is loaded directly into it.
- If RAM is full:
  The OS must select a page to remove using a page replacement algorithm such as FIFO, LRU, or Optimal.
- The algorithm is only needed when there is no free frame.

6. Required Page is Loaded:

Page 5 is transferred from the SSD into the selected frame in RAM.

7. Page Table is Updated:

The OS updates the page table.

For example:

Page 5 → Frame 2

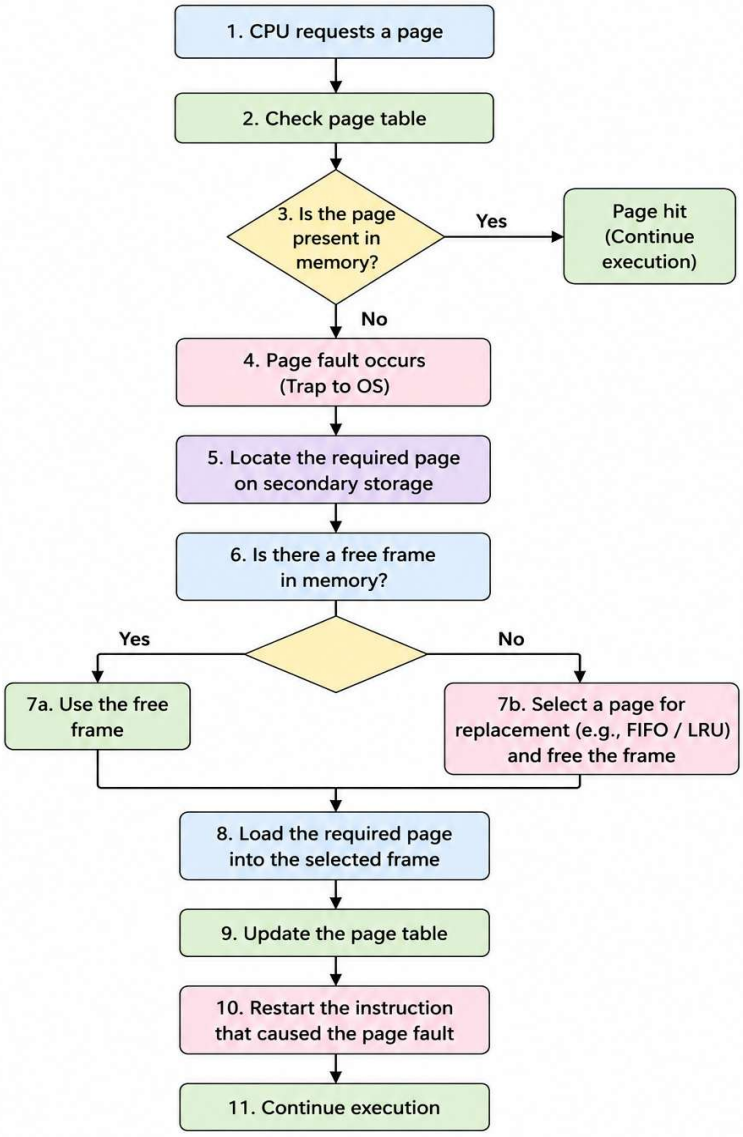
Valid Bit → 1

Now the system knows that Page 5 is available in RAM.

8. Instruction is Restarted:

- The CPU restarts the instruction that caused the page fault.
- This time, the required page is found in RAM, so execution continues normally.

Page Fault Handling Process



3. Explain the need for **page replacement algorithms** in virtual memory systems. Describe the working of the **FIFO page replacement algorithm** with a suitable example.

CASE STUDY: Page Replacement Algorithms

Situation:

Suppose a computer has **3 frames in RAM**, but a program needs more than 3 pages.

For example, the program requests pages in this order:

1 → 2 → 3 → 4

Initially:

RAM

| Page 1 | Page 2 | Page 3 |

Now the program requests **Page 4**.

Page 4 is not in RAM, so a **page fault** occurs. But all 3 frames are already occupied.

So the operating system needs to answer:

"Which page should I remove to bring Page 4 into memory?"

This is why we need a page replacement algorithm.

Why Do We Need Page Replacement Algorithms?

When a page fault occurs and **RAM has no free frame**, the OS must remove one existing page and load the required page.

A page replacement algorithm decides which page should be removed.

A good algorithm should:

- Reduce the number of page faults.
- Improve memory utilization.
- Reduce unnecessary disk access.
- Improve overall system performance.

Common algorithms are:

FIFO, LRU, Optimal, LFU, etc.

FIFO Page Replacement Algorithm

FIFO = First In, First Out

The basic idea is very simple:

Example

Assume:

- Number of frames = **3**
- Page reference string = **1, 2, 3, 4, 1, 5**

Let's process them one by one.

Step 1: Page 1

Page 1 is not in memory → **Page Fault**

| 1 | | |

Step 2: Page 2

Page 2 is not in memory → **Page Fault**

| 1 | 2 | |

Step 3: Page 3

Page 3 is not in memory → **Page Fault**

| 1 | 2 | 3 |

Now all frames are full.

Step 4: Page 4

Page 4 is not in memory → **Page Fault**.

Which page entered first?

Page 1

So FIFO removes Page 1 and inserts Page 4.

Before:

| 1 | 2 | 3 |

After:

| 4 | 2 | 3 |

Step 5: Page 1

Page 1 is no longer in memory → **Page Fault**.

The next oldest page is **Page 2**, so FIFO removes Page 2.

| 4 | 1 | 3 |

Step 6: Page 5

Page 5 is not in memory → **Page Fault**.

The next oldest page is **Page 3**, so FIFO removes Page 3.

| 4 | 1 | 5 |

Complete Table:

Page Request	Frame 1	Frame 2	Frame 3	Result
1	1	-	-	Fault
2	1	2	-	Fault
3	1	2	3	Fault
4	4	2	3	Fault
1	4	1	3	Fault
5	4	1	5	Fault

Total Page Faults = 6

4. Discuss the concepts of **linking and loading** in detail. Explain the differences between static linking and dynamic linking, and describe how loading prepares a program for execution.

CASE STUDY: Linking and Loading

Situation:

- Suppose a student writes a C program to calculate the total marks of a student. The program contains functions such as printf() and scanf(), which are provided by system libraries.
- When the student compiles the program, the computer cannot directly execute the source code. The program has to go through **compilation, linking, and loading** before execution.

I. Linking

- **Linking** is the process of combining the object code generated by the compiler with the required library code to create an **executable file**.
- For example, if a program uses a library function such as printf(), the linker makes sure that the required function is available to the program.
- The linker also resolves references to functions and variables used in the program.

Example:

A student writes:

main.c

After compilation:

main.o

The linker combines the object file with the required libraries and produces:

program.exe

II. Static Linking

- In **static linking**, the required library code is copied directly into the executable file during the linking process.

- For example, if a program needs a particular library function, the required code becomes part of the final executable.

Advantages:

- The executable contains all required library code.
- The program does not depend heavily on external library files during execution.
- It is easier to distribute the complete program.

Disadvantages:

- Executable files are larger.
- The same library code may be stored separately in many programs.
- If a library is updated, the program may need to be rebuilt.

III. Dynamic Linking

- In **dynamic linking**, library code is not copied completely into the executable. Instead, the program connects to a **shared library** when the library is needed.
- For example, several programs can use the same shared library instead of each program having its own copy.

Advantages:

- Smaller executable files.
- Libraries can be shared by multiple programs.
- Updating a shared library can update its functionality for multiple applications.

Disadvantages:

- The required library must be available on the system.
- If the library is missing or incompatible, the program may not run correctly.
- There can be some additional overhead when the library is loaded.

IV. Difference Between Static and Dynamic Linking

Static Linking	Dynamic Linking
Library code is included in the executable.	Library code is kept separately.
Produces a larger executable.	Produces a smaller executable.
Less dependency on external libraries during execution.	Depends on shared libraries.
Libraries are not easily shared between programs.	Libraries can be shared by many programs.
Changes usually require rebuilding the program.	Shared libraries can be updated separately.

V. Loading

- After linking, we have an executable file. But the CPU cannot directly execute a program stored on the hard disk or SSD.
- **Loading** is the process of bringing the executable program from secondary storage into **main memory (RAM)** so that the CPU can execute it.

The **loader** performs several important tasks:

- Loads the program into RAM.
- Allocates the required memory.
- Performs address relocation if necessary.
- Sets up the program's execution environment.
- Transfers control to the program's starting instruction.

Simple Example:

- Suppose program.exe is stored on the SSD.
- When the user double-clicks the program, the operating system's loader loads the required parts of the program into RAM and prepares them for execution.
- The CPU can then start executing the program.

VI. Why Linking and Loading Are Important

- Linking and loading are important because a program written by a programmer is not immediately ready for CPU execution.
- **Linking** makes the executable program complete by resolving required functions and libraries.
- **Loading** prepares that executable program in memory so that the CPU can execute it.

---

5. Analyze how the combination of **paging, segmentation, TLB, and virtual memory** improves system performance and memory utilization in the given case study. Provide a detailed explanation.

CASE STUDY: Combining Paging, Segmentation, TLB and Virtual Memory

Situation:

- Consider a student using a computer with 4 GB of RAM. The student opens Chrome, Android Studio, MS Word, and a music player at the same time.
- These applications together may require more memory than the available physical RAM. The operating system uses virtual memory, paging, segmentation, and TLB together to manage memory efficiently.

I. Virtual Memory

Virtual memory allows the system to run programs that require **more memory than the available RAM**.

For example:

- Available RAM = **4 GB**
- Total memory required by applications = **6 GB**

The operating system does not need to keep all 6 GB in RAM at once. It keeps the currently required portions in RAM and stores less-used portions on the SSD/HDD.

This allows more applications to run simultaneously.

Benefit: Better memory utilization and support for large programs.

II. Paging

Paging divides the virtual memory of a process into fixed-size **pages** and physical memory into fixed-size **frames**.

For example, if a program has 10 pages but only 4 frames are currently available, the OS can keep only the required pages in RAM.

When a required page is not in RAM, a **page fault** occurs, and the OS brings that page from secondary storage.

Benefit:

- Reduces external fragmentation.
- Uses available RAM efficiently.
- Allows programs larger than physical memory to execute.

III. Segmentation

Segmentation divides a program according to its logical parts, such as:

- **Code segment** – contains program instructions.
- **Data segment** – contains variables and data.
- **Stack segment** – contains function calls and local variables.

For example, the operating system can provide different protection for the code and data sections.

The code segment can be made **read-only**, preventing accidental modification of program instructions.

Benefit:

- Provides logical organization of programs.
- Supports memory protection.
- Allows different sections to have different access permissions.

In many modern systems, paging does most of the practical memory-management work, while segmentation is limited or used mainly for protection/compatibility.

IV. Translation Lookaside Buffer (TLB)

Paging requires the system to use a **page table** to translate virtual page numbers into physical frame numbers.

Checking the page table for every memory access would take additional time.

The **TLB** solves this problem by storing recently used page-to-frame translations.

For example:

Page 5 → Frame 12

Page 8 → Frame 4

Page 2 → Frame 9

If the CPU requests Page 5 again, the TLB can provide the frame number quickly without requiring a full page-table lookup.

Benefit: Faster address translation and reduced memory access time.

How They Work Together?

In this case study, each technique performs a different job:

Technique	Main Purpose	Benefit
Virtual Memory	Gives programs a larger logical memory space	Runs large programs
Paging	Divides memory into pages and frames	Efficient RAM usage
Segmentation	Organizes memory into logical sections	Protection and organization
TLB	Stores recent address translations	Faster memory access

Overall Example

- Suppose the student is working in Android Studio.
- The CPU generates a virtual address. The memory-management system identifies the relevant **segment** and **page**. The TLB is checked first for the recent page-to-frame mapping.
- If the mapping is found in the TLB, the physical address can be obtained quickly.
- If it is not found, the page table is checked.
- If the required page is not in RAM, a **page fault** occurs and the operating system loads the page from secondary storage.
- Thus, all four techniques contribute to the system:
    - **Virtual memory** → allows more memory to be used than physical RAM.
    - **Paging** → manages RAM efficiently.
    - **Segmentation** → provides logical organization and protection.
    - **TLB** → makes address translation faster.

Overall Analysis

- The combination of these techniques improves both **performance and memory utilization**.
- **Memory utilization improves** because paging allows only the required pages to occupy RAM, while virtual memory allows less-used pages to remain on secondary storage.
- **Performance improves** because the TLB avoids repeated page-table lookups for frequently accessed pages.
- **Security and protection improve** because segmentation can separate code, data, and stack areas and provide different access permissions.

However, virtual memory can also reduce performance if there are too many page faults because SSD/HDD access is much slower than RAM. Therefore, the operating system must manage pages efficiently and keep frequently used pages in memory.